

Chapter 1

Introduction

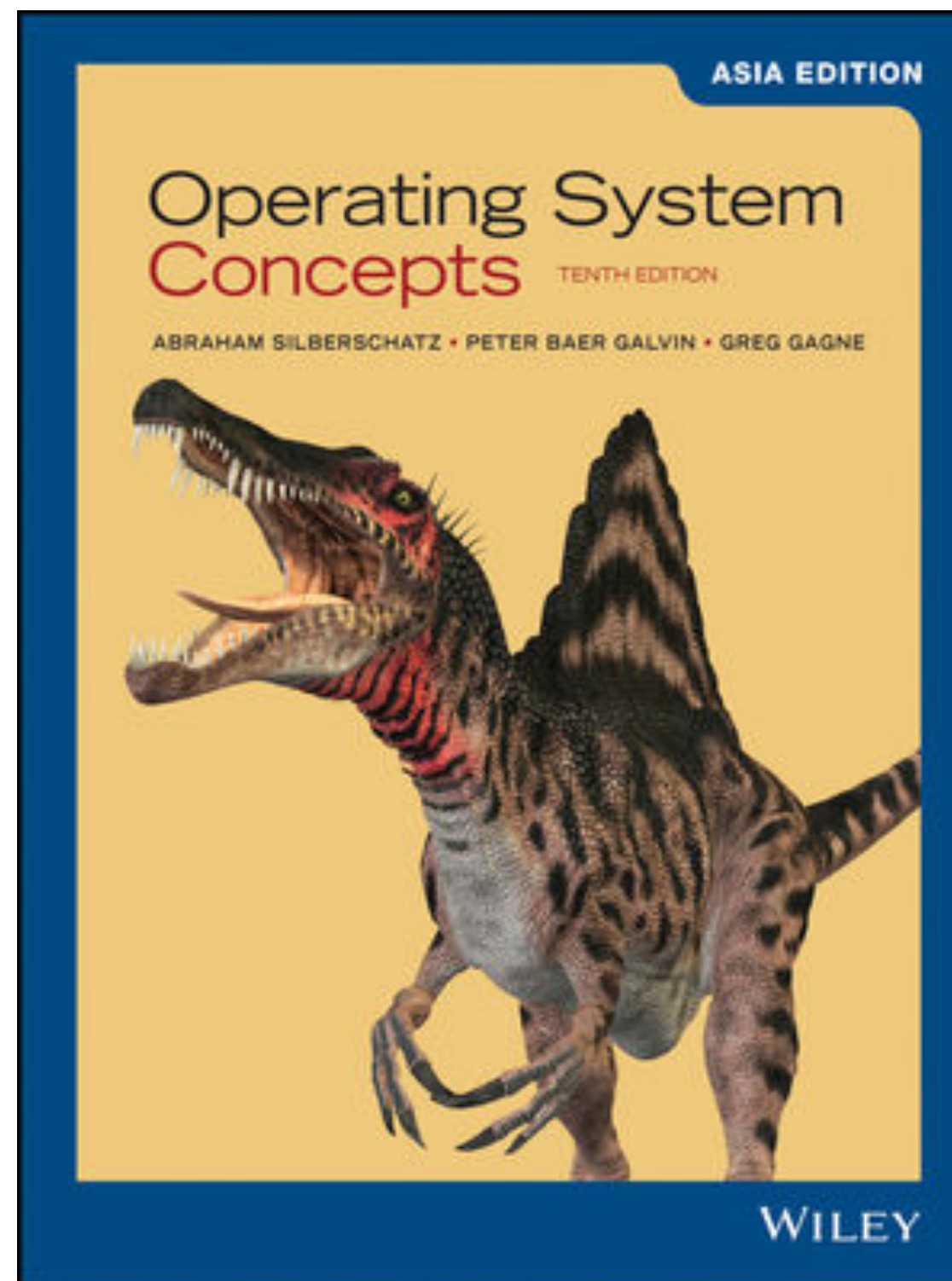
Chih-Yu Lin

National Taiwan Ocean University

Updated date: 2025/7/8

Textbook

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, "Operating System Concepts, Asia Edition," 10/e, ISBN: 978-1-119-58616-6, Wiley



Outline

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Operations
- Resource Management
- Virtualization

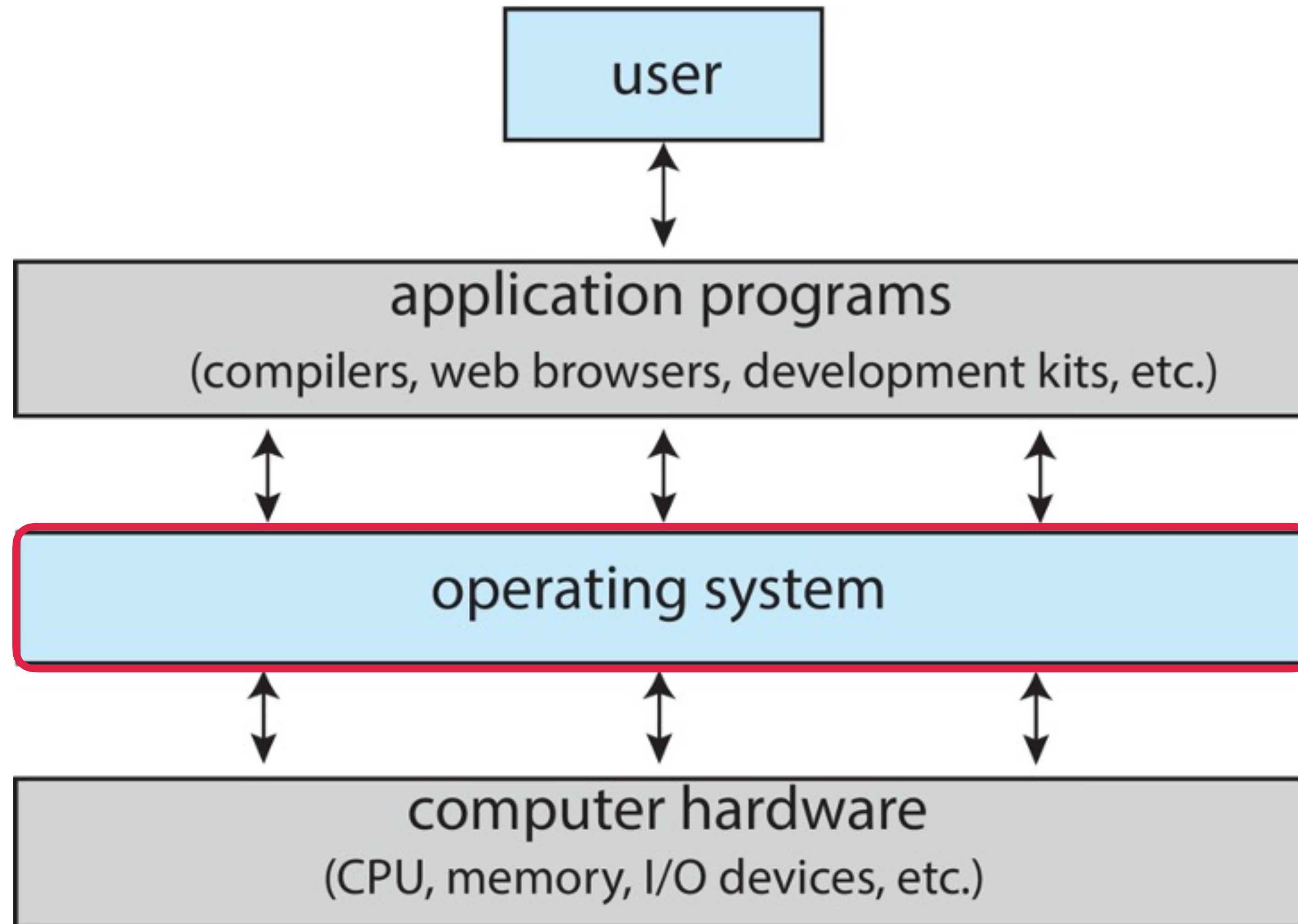
Terminology

- Computer-System Organization
 - Interrupt / Context Switch
- Computer-System Architecture
 - SMP / NUMA
- Operating-System Operations
 - Multitasking (Timesharing) / Dual-mode (User/Kernel)
- Resource Management
- Virtualization

Outline

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Operations
- Resource Management
- Virtualization

Abstract View of Components of Computer



Computer System Structure

作業系統扮演了協調、調度資源的角色

- A computer system can be divided into four components:
 - **Users**
 - People, machines, other computers
 - **Application programs** – define the ways in which the system resources are used to solve the computing problems of the users
 - Word processors, compilers, web browsers, database systems, video games
 - **Operating system**
 - Controls and coordinates use of hardware among various applications and users
 - **Hardware** – provides basic computing resources
 - CPU, memory, I/O devices

What is an Operating System?

- A program that acts as an intermediary (中介) between a user of a computer and the computer hardware
- Operating system goals
 - Execute user programs and make solving user problems easier
 - Make the computer system convenient to use
 - Use the computer hardware in an efficient manner

Operating System's Role

- Depends on the point of view
 - ／ 个人电脑强调的特質：易於始用 & 高效能
- Users want convenience, **ease of use** and **good performance**
- **Don't care about resource utilization** 較不在意資源的使用率(相較之下)
 - 合理、公平的將資源配給所有使用者！
- But shared computer such as **mainframe** or **minicomputer** must keep **all users** happy
 - Operating system is a **resource allocator** and **control program** making efficient use of HW and managing execution of user programs
- Mobile devices like smartphones and tables are resource poor, optimized for usability and **battery life**
 - ／ 人機介面(触控螢幕、聲紋辨識)
 - Mobile user interfaces such as **touch screens**, voice recognition
- Some computers have **little or no user interface**, such as **embedded computers** in devices and automobiles
 - Run primarily **without user intervention**

Operating System Definition

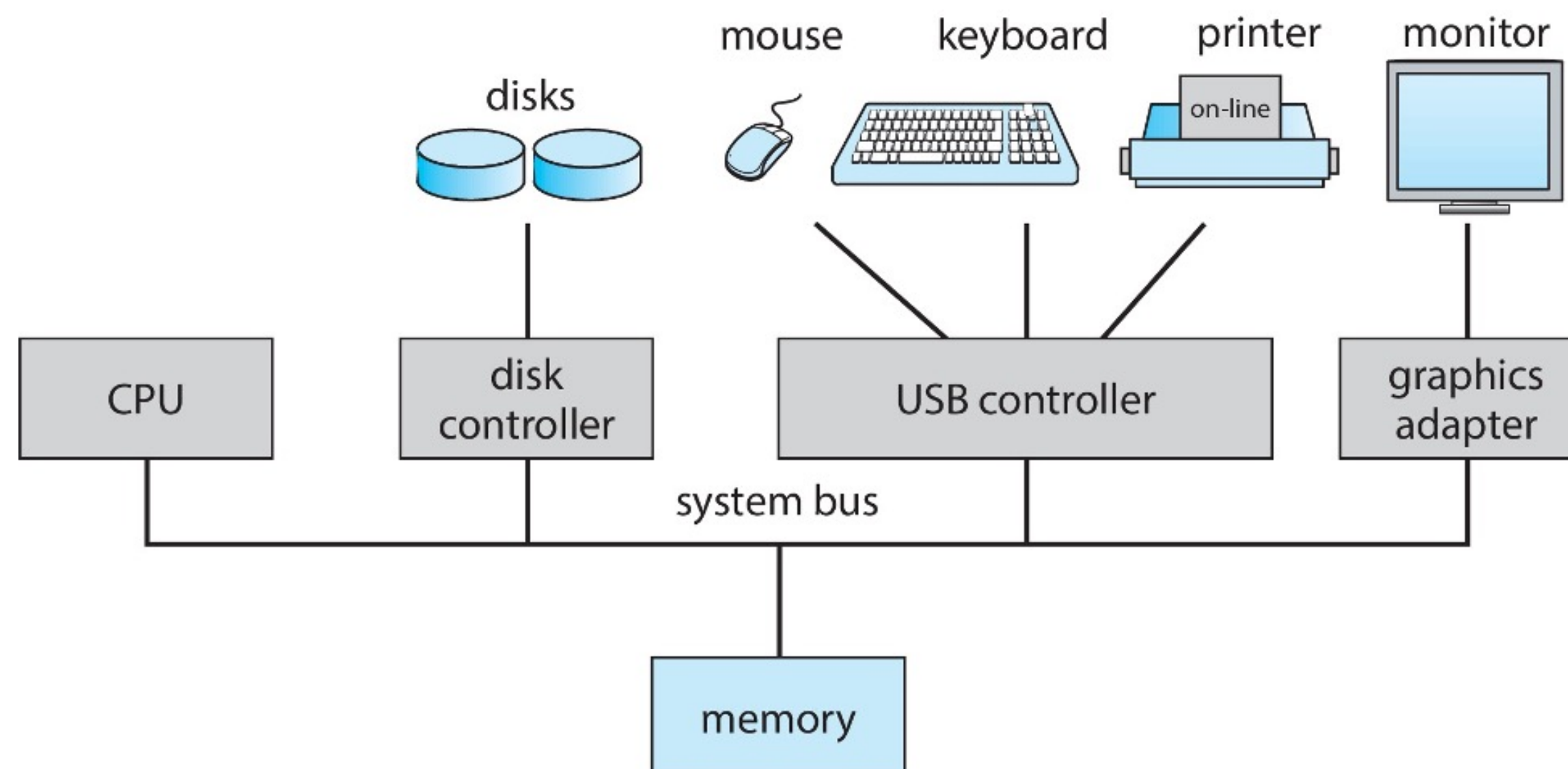
- No universally accepted definition
“作業系統”沒有一個廣泛、可被接受的定義
- “The one **program** running at all times on the computer”- usually called the **kernel**
- Everything else is either
——有些人會認為 System Program (e.g. Loader) 也算是系統程式, 有些人則不然
 - A **system program** (ships with the operating system, but not part of the kernel), or
 - An **application program**, all programs not associated with the operating system

Outline

- What Operating Systems Do
- Computer-System Organization (Interrupt / Context Switch)
- Computer-System Architecture
- Operating-System Operations
- Resource Management
- Virtualization

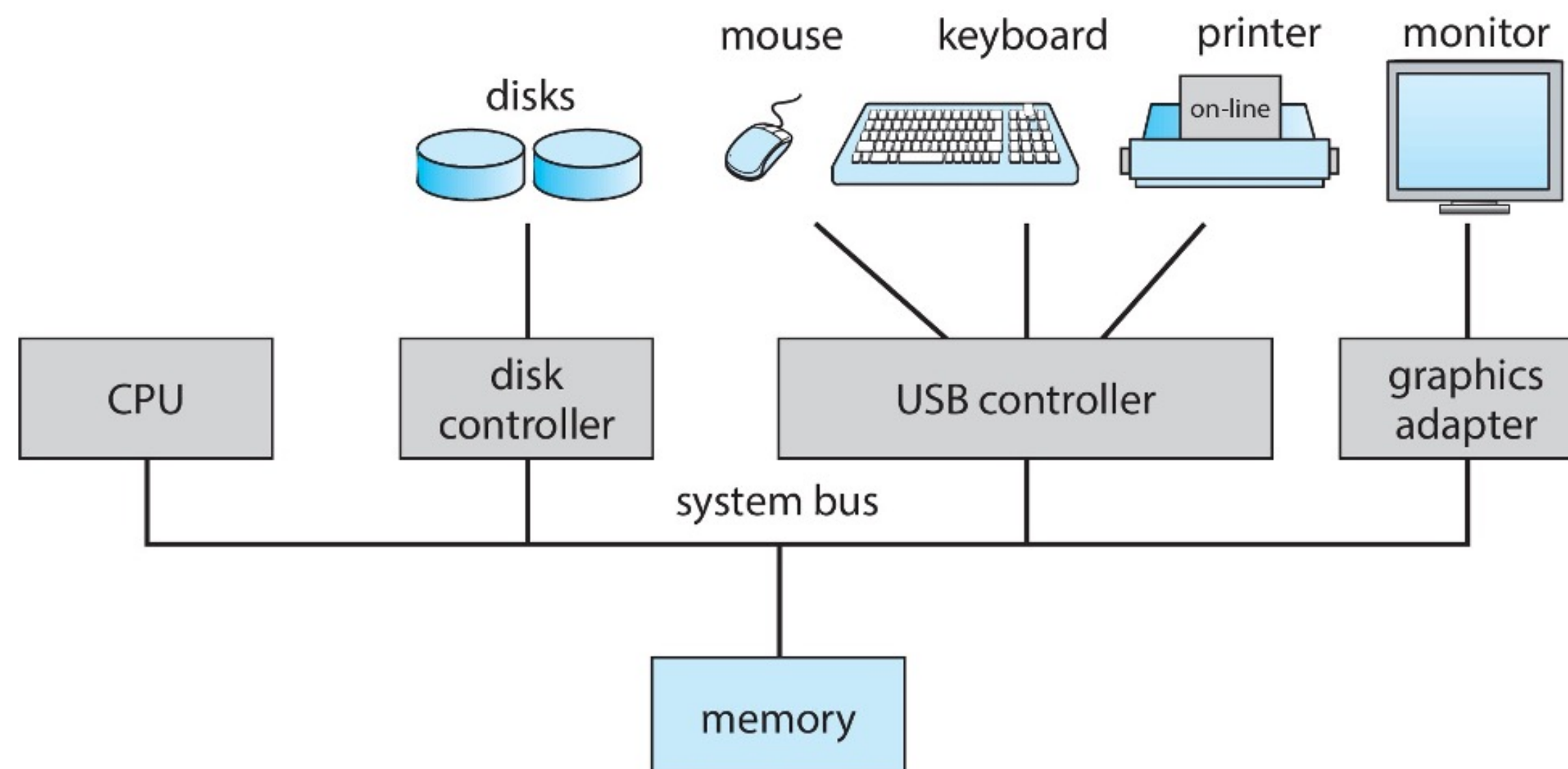
Computer System Organization

- A modern ^{"通用類形"} *vs. special-purpose* general-purpose computer system
- One or more CPUs, device controllers connect through **common bus** ^{通用串行總線} providing access to **shared memory**
- Concurrent execution of CPUs and devices competing for memory cycles
- The term "**cycles**" refers to the time units needed by the memory to complete one access operation.



Computer System Organization

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- A **device controller** maintains some **local buffer storage** and **a set of special-purpose registers**
 - e.g. program counter registers.*
 - 本机缓冲区*
- A device controller is an **electronic hardware component**, typically a small **specialized circuit board or chip connected to the motherboard** or inserted as an expansion card.
- Each device controller type has an operating system **device driver** to manage it



Computer System Operation

- CPU moves data from/to main memory to/from local buffers
 - **Direct Memory Access (DMA)** 记忆体直接存取 (不通过CPU)
 - The device controller transfers an entire block of data directly to or from the device and main memory
- I/O is from the device to local buffer of controller
- Device controller informs CPU that it has finished its operation by causing an **interrupt**

CPU & I/O 互动的方法:

- Polling (輪巡)
- Interrupt

1. 程序直接控制方式(Program-Controlled I/O)

⇒ Polling (輪巡)

- 原理: CPU 通过不断轮询 I/O 控制器的状态寄存器, 来判断设备是否准备好进行数据传输, 然后 CPU 自己负责读写数据, 效率低下。
- 适用场景: 适用于简单的、外设很少的计算机系统。

2. 中断控制方式(Interrupt-Driven I/O)

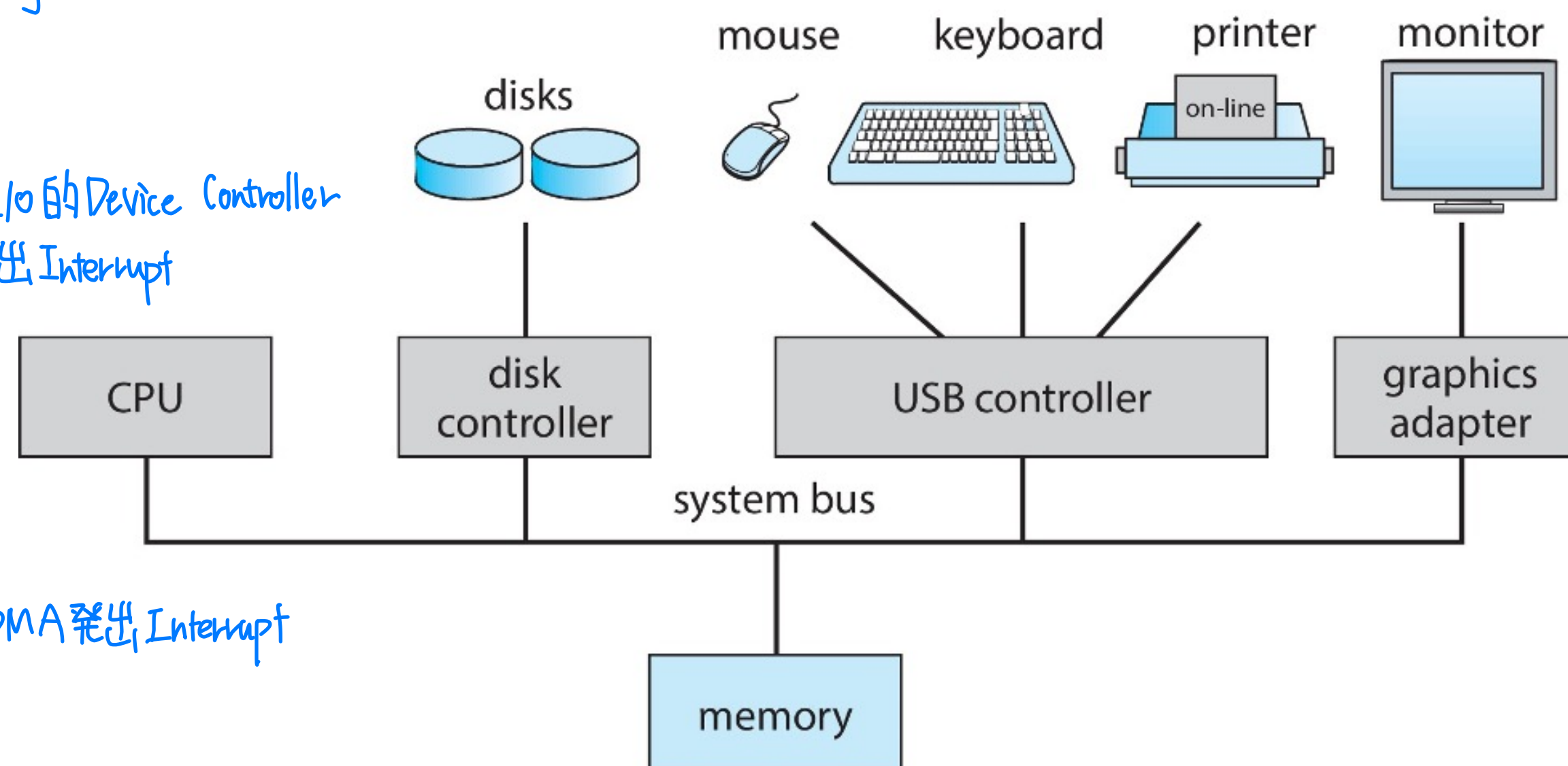
→ 由 I/O 的 Device Controller 发出 Interrupt

- 原理: CPU 初始化一次 I/O 操作后, 就去执行其他任务。当 I/O 设备完成数据传输后, 会向 CPU 发送一个中断信号, 请求 CPU 来处理。
- 优点: 相比程序直接控制方式, CPU 的时间利用率更高。
- 缺点: 如果中断次数过多, CPU 仍然需要花费较多时间来处理中断, 并且对并行操作的设备台数有限制。

3. 直接存储器访问(DMA) 方式

→ 由 DMA 发出 Interrupt

- 原理: CPU 发出 I/O 指令后, 将数据传输的工作完全交给 DMA 控制器, 由 DMA 控制器直接在 I/O 设备和主存之间进行数据传输。
- 优点: 大大减轻了 CPU 的工作负担, 提高了 CPU 的利用率, CPU 只需要在传输开始时发出指令, 结束后接收 DMA 控制器发出的中断信号进行后续处理。
- 缺点: 数据块的传输需要是连续的, 并且需要 CPU 在开始前给出数据在内存和设备中的具体位



Direct Memory Access

Advantages of DMA

The primary benefit of DMA is CPU resource liberation. During large data transfers, the CPU can perform other tasks instead of handling each byte transfer individually, significantly improving overall system performance. This is particularly crucial for high-speed I/O operations such as disk reads/writes and network transfers.

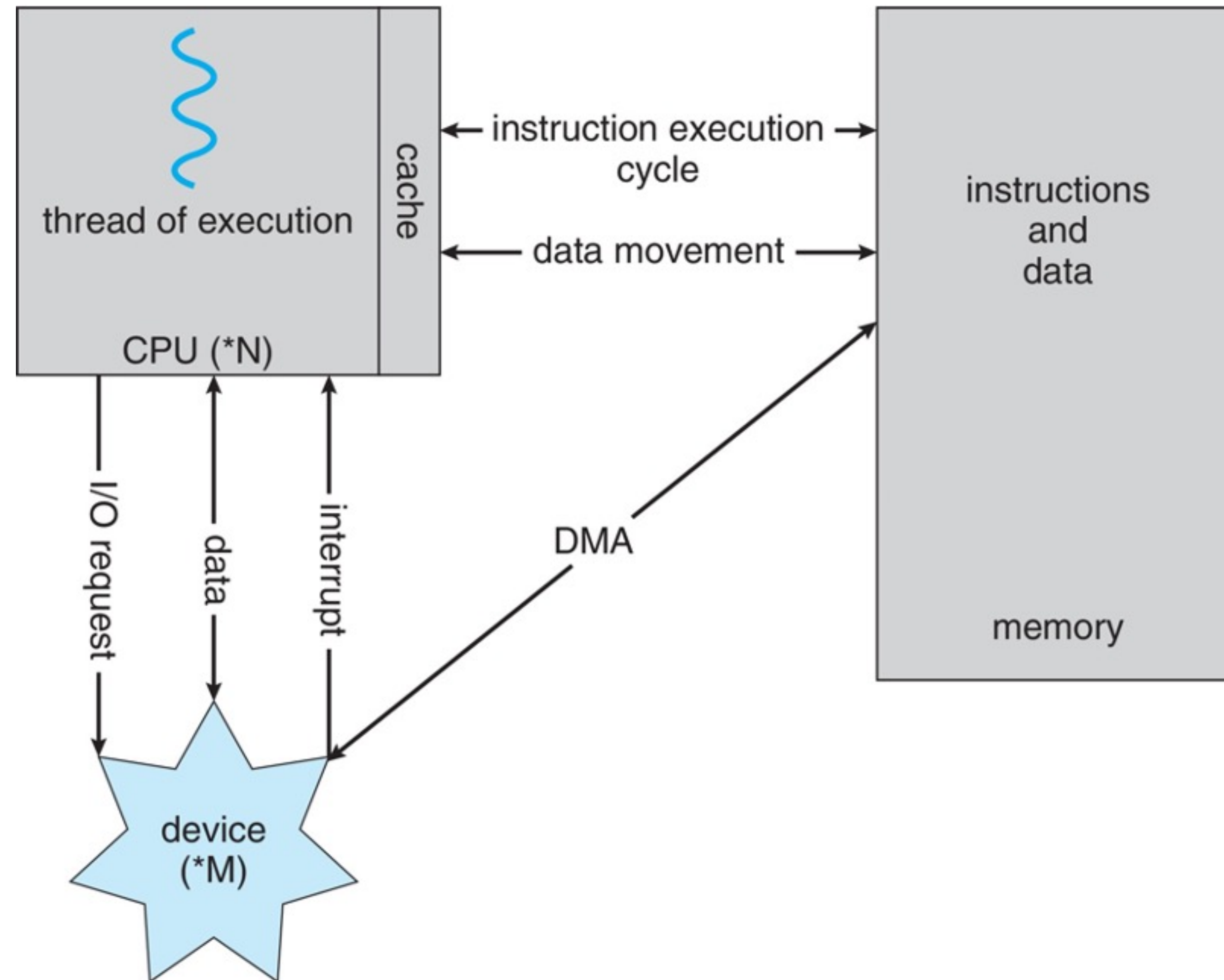
Alternatives Without DMA

Programmed I/O (PIO): This is the most basic approach where the CPU must personally handle every data transfer. The CPU continuously polls the device status and performs data read/write operations when the device is ready. This method consumes substantial CPU time, especially when processing large files or high-bandwidth requirements.

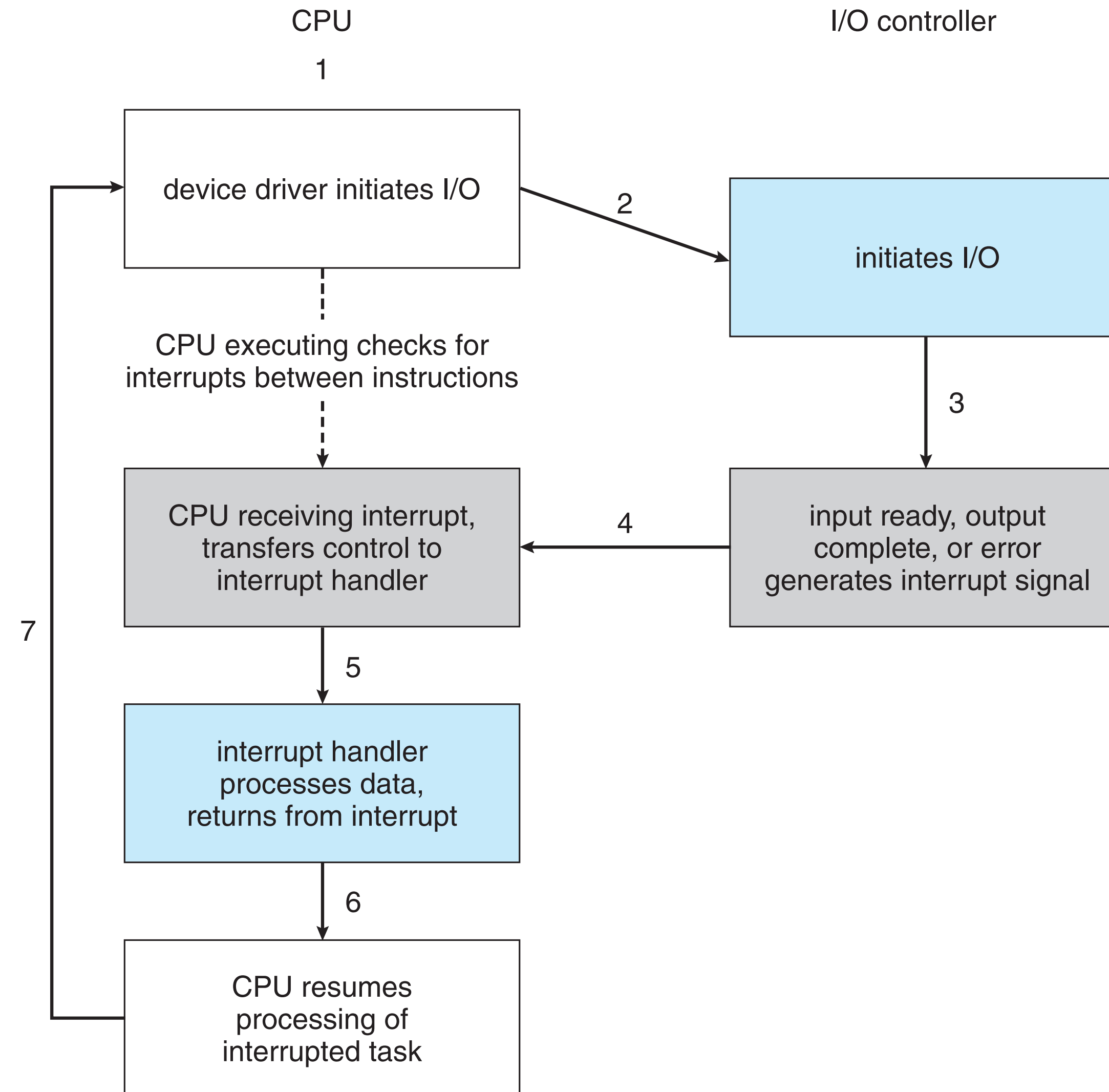
Interrupt-driven I/O: This improves upon the efficiency issues of programmed I/O. When an I/O device is ready for data transfer, it sends an interrupt signal to the CPU. The CPU responds to the interrupt, handles the data transfer, then continues with other tasks. While more efficient than polling, the CPU still participates in every data transfer.

Memory-mapped I/O: This maps I/O device control registers into the memory address space, allowing the CPU to operate I/O devices like accessing regular memory. This simplifies programming but still requires direct CPU involvement in data processing.

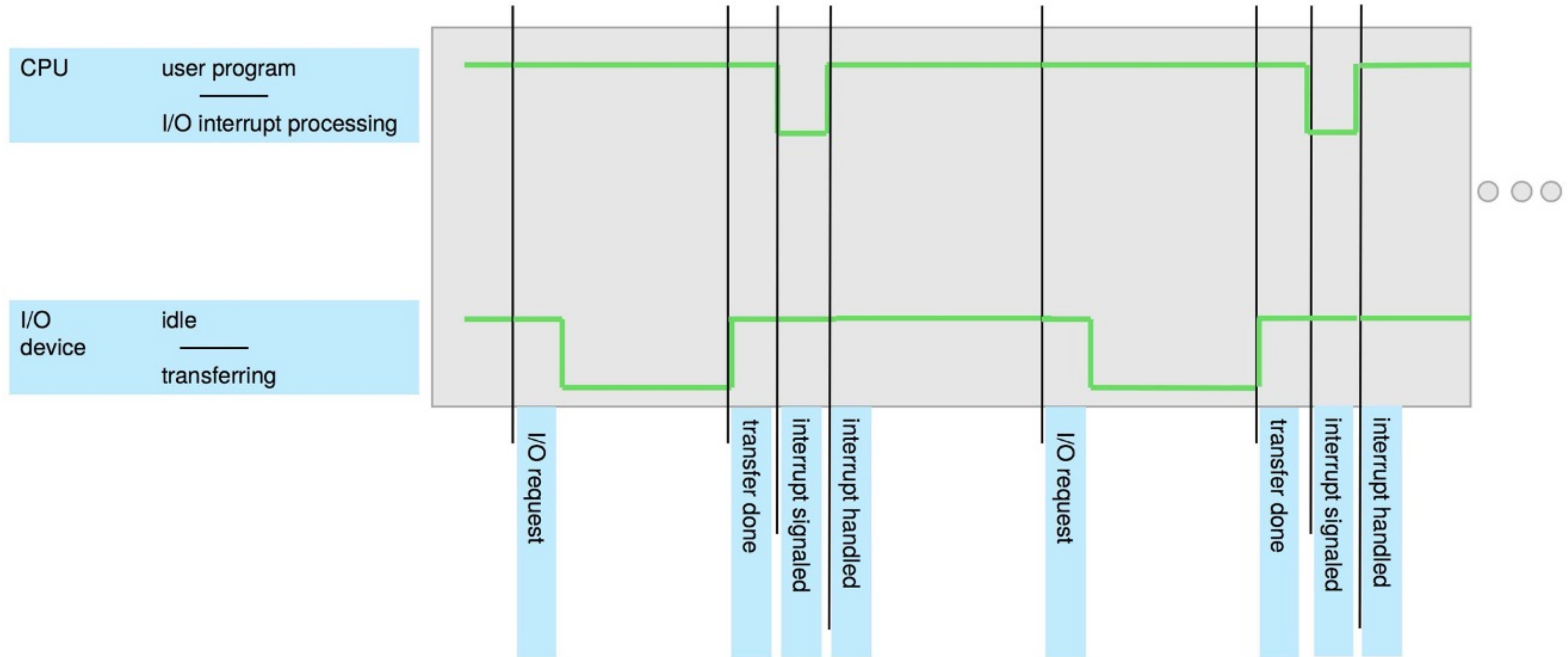
How a Modern Computer Works



Interrupt-driven I/O Cycle



Interrupt Timeline



Interrupt Vector

一个由 OS 維護的 Table, 負責記錄 interrupt handler 的記憶體位置

- An interrupt vector is **the memory location of an interrupt handler**, which prioritizes interrupts and saves them in a queue if more than one interrupt is waiting to be handled.
- An interrupt is a signal from a device attached to a computer, or from a program within the computer, that tells the OS (operating system) to stop and decide what to do next. When an interrupt is generated, the OS saves its execution state by means of a **context switch**, a procedure that a computer processor follows to change from one task to another while ensuring that the tasks do not conflict. Once the OS has saved the execution state, it starts to execute the **interrupt handler** at the interrupt vector.
(interrupt service routine, ISR)
CPU 會先去 Interrupt Vector 找下一個切換的 Process, 如果下一個 Process 是 另一個 Process: 發生 Context Switch 還是原 Process 只會有 Context Saving 發生 (Context Switch 前半部分, 將當前程式狀態存起來)

發生 Interrupt 時, 會連帶觸發 Context Switch, 先把當下的狀態 (register / mem 的值) 儲存起來, 然後先

去處理 Interrupt, 當 Interrupt 處理完成, 在進行一次 Context Switch, 將剛剛記錄的狀態載入回來。

↳ 這個儲存、載入的動作, 即是 Context Switch (上下文切換)
上文為 Save Cond.、下文為 Load Cond.

Context Switch

Context switches occur in several scenarios within an operating system:

1. Multitasking and Process Scheduling

- When a process's time slice expires (in time-sharing systems)
- When a higher-priority process becomes ready to run
- When the scheduler decides to implement its scheduling algorithm

2. Interrupt Handling

- Hardware interrupts (I/O completion, network packet arrival, etc.)
- Timer interrupts that signal the end of a time quantum
- Device interrupts requiring immediate attention

Interrupt Driven

- An operating system is interrupt driven
 - **Hardware** interrupt by one of the devices
 - **Software** interrupt (**exception** or **trap**):
 - Software error (e.g., division by zero)
 - Request for operating system service – system call

Outline

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Operations
- Resource Management
- Virtualization

Definitions of Computer System Components

DEFINITIONS OF COMPUTER SYSTEM COMPONENTS

- **CPU**—The hardware that executes instructions. → 單指中央處理器
- **Processor**—A physical chip that contains one or more CPUs. → 泛指任何能執行運算或處理資料的單元。
- **Core**—The basic computation unit of the CPU.
- **Multicore**—Including multiple computing cores on the same CPU.
- **Multiprocessor**—Including multiple processors.

Although virtually all systems are now multicore, we use the general term *CPU* when referring to a single computational unit of a computer system and *core* as well as *multicore* when specifically referring to one or more cores on a CPU.

Single-Processor Systems

- The **core** is the component that executes instructions and registers for storing data locally.
- The one main CPU with its core is capable of executing a general-purpose instruction set

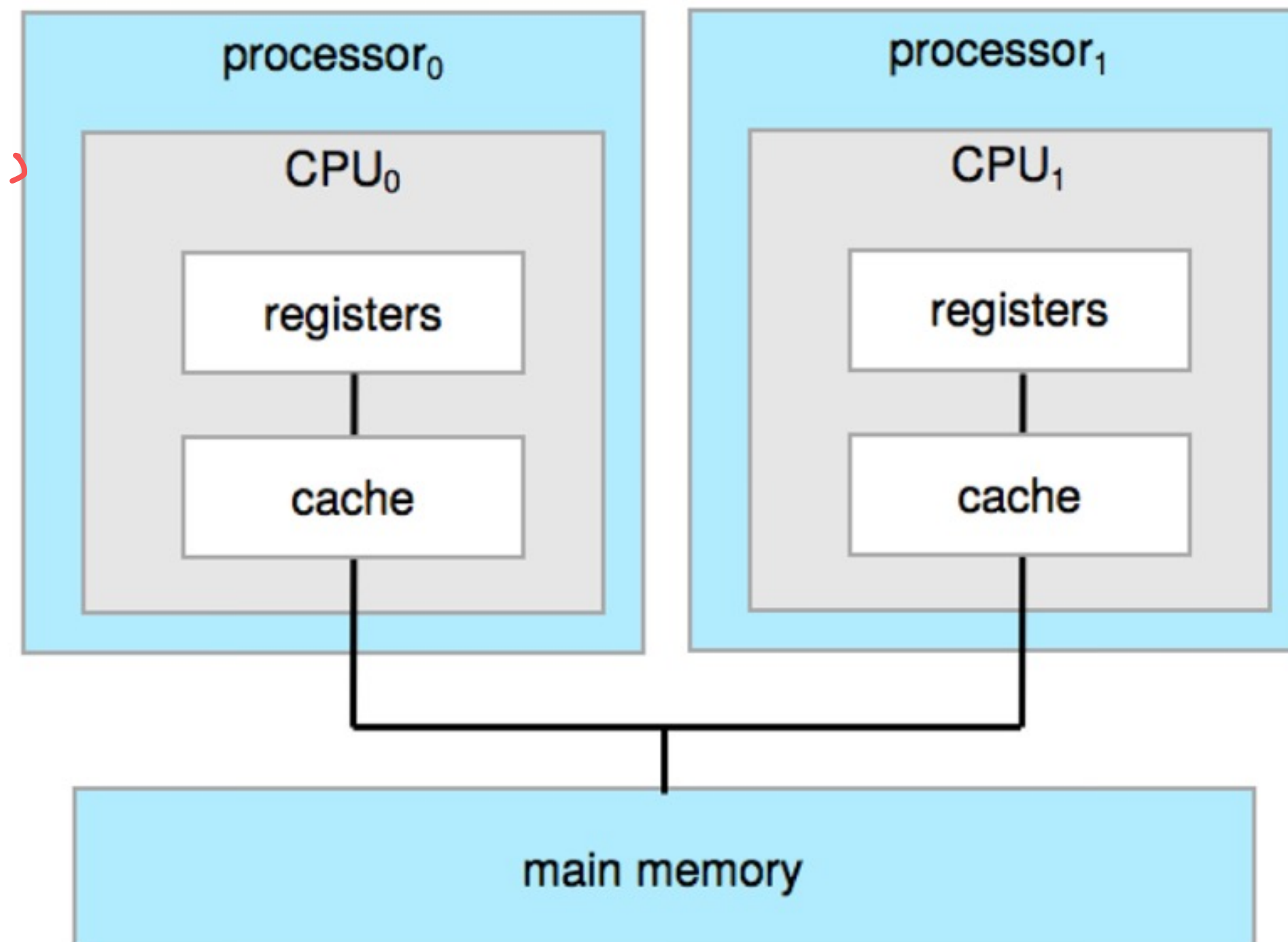
Multiprocessor Systems

- On modern computers, from **mobile devices** to servers, multiprocessor systems now dominate the landscape of computing
- The definition of **multiprocessor** has evolved over time and now includes **multicore** systems, in which multiple computing cores reside on a **single chip**
- The most common multiprocessor systems use **symmetric multiprocessing (SMP)**
 - https://en.wikipedia.org/wiki/Symmetric_multiprocessing

Symmetric MultiProcessing architecture (SMP)

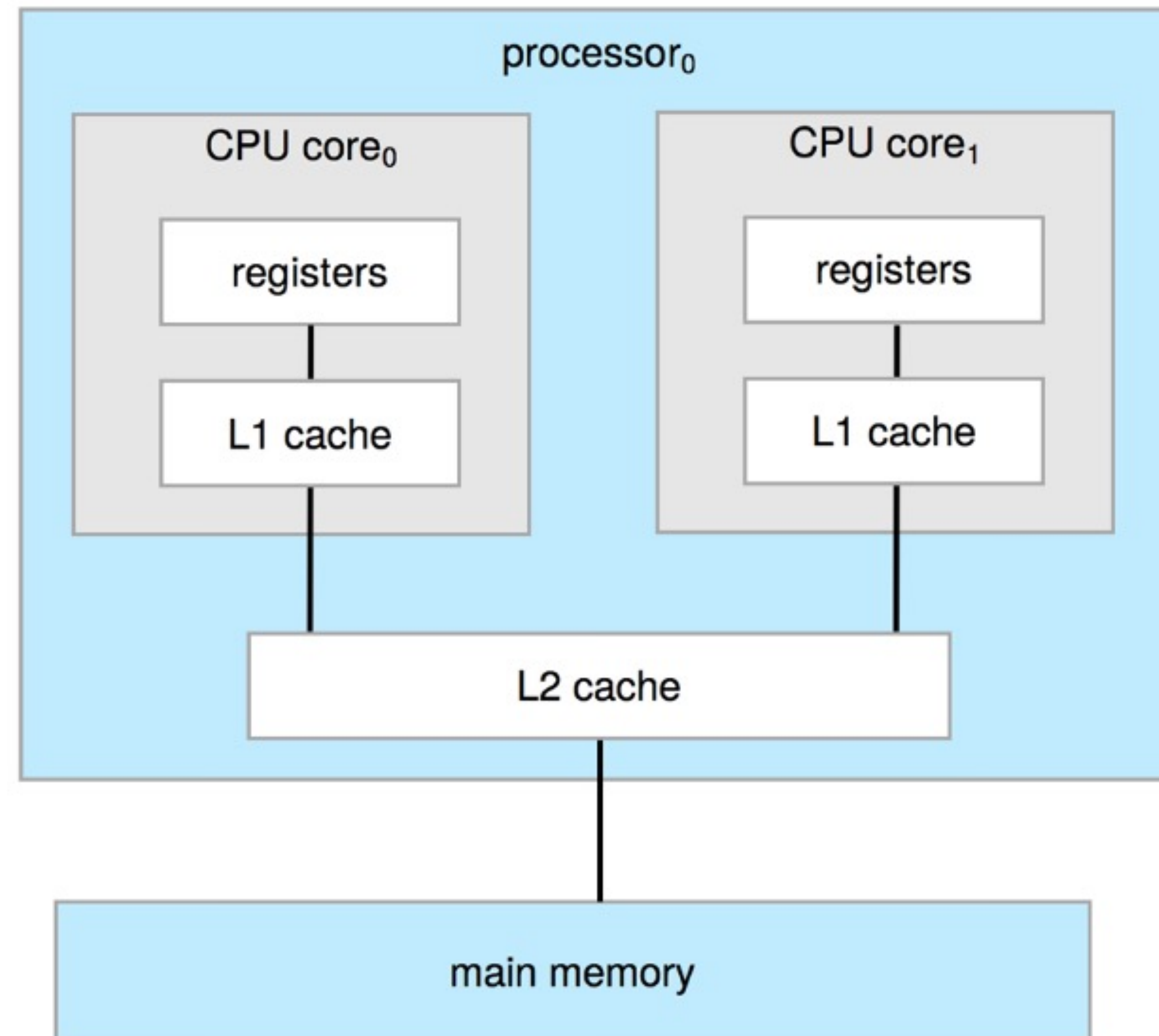
- Asymmetric Multiprocessing – each processor is assigned a specific task.
- **Symmetric Multiprocessing – each processor performs all tasks**
 - Virtually all modern operating systems—including Windows, macOS, and Linux, as well as Android and iOS mobile systems—support multicore SMP systems.

SMP架構下，所有的Processor
都可以獨立運作(不用互相協作)，
共享同記憶體與其他資源，在
同OS下平行執行



Dual-core Design

- A dual-core design with two cores on the same chip

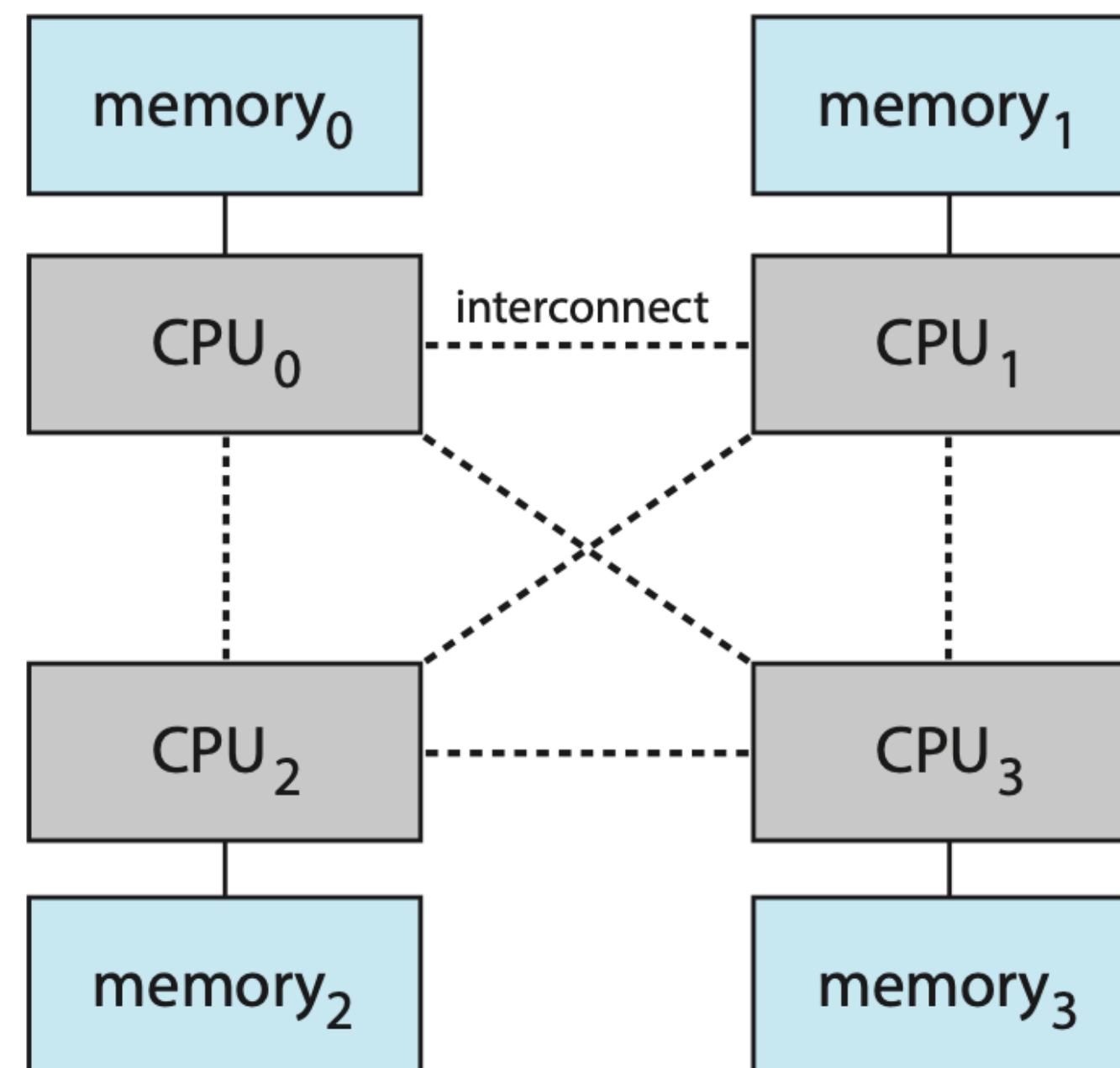


Non-Uniform Memory Access (NUMA)

解決 SMP 架構的記憶體競爭問題

- Once we add too many CPUs, contention for the system bus becomes bottleneck and performance begins to degrade
- Provide each CPU with its own local memory
- The CPUs are connected by a shared system interconnect

系統互聯

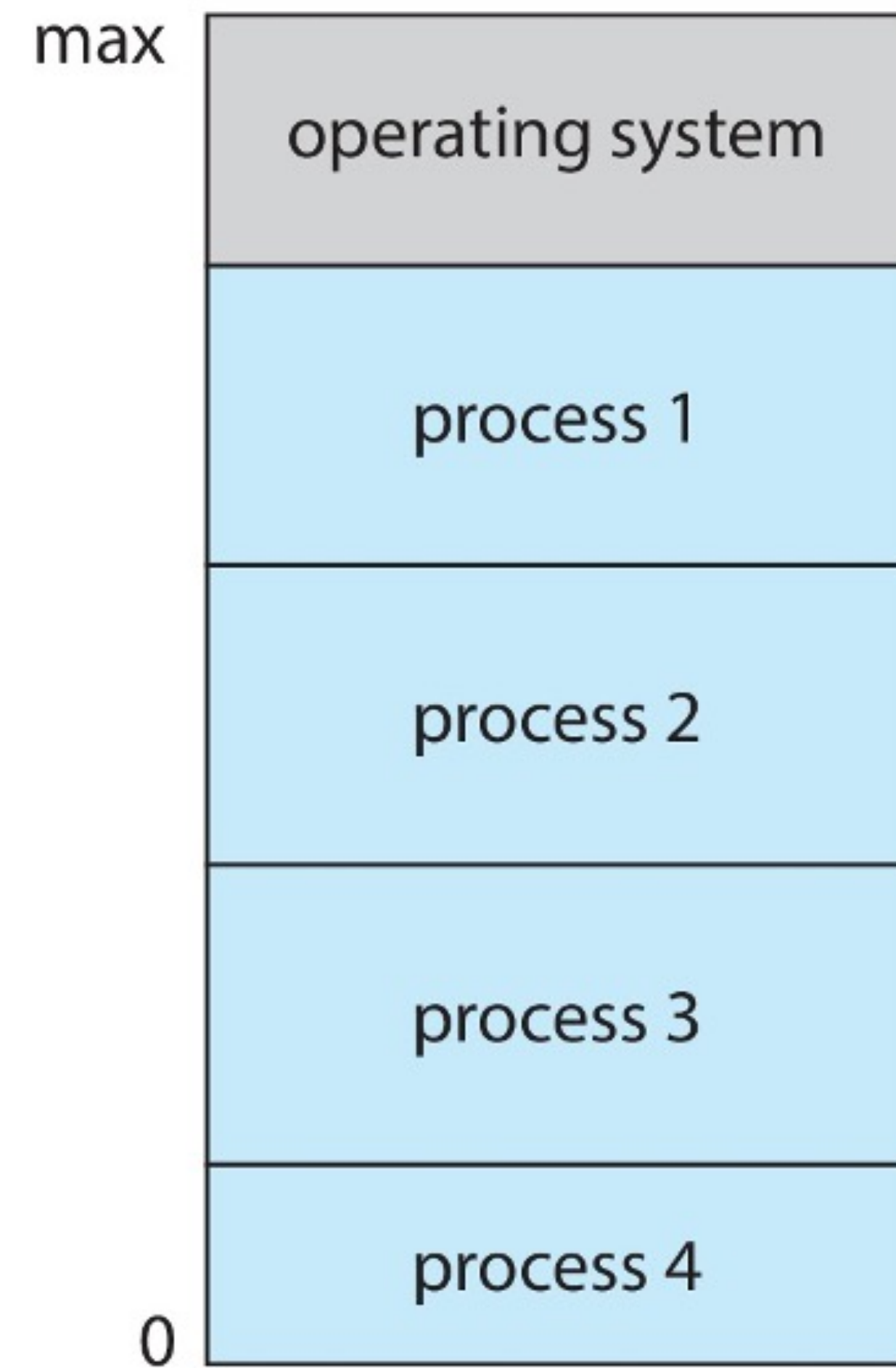


Outline

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Operations
 - Multitasking (Timesharing) / Dual-mode (User/Kernel)
- Resource Management
- Virtualization

Multiprogramming

- Single user cannot always keep CPU and I/O devices busy
- Multiprogramming organizes jobs (code and data) so CPU always has one to execute (**increases CPU utilization**)
- A subset of total jobs in system is kept in memory
- One job selected and run via job scheduling
- When a job has to wait (for I/O for example), OS switches to another job
- In a multiprogramming system, a program in execution is termed a **process**



Multitasking (Timesharing)

- A logical **extension** of Batch systems– the CPU **switches** jobs **so frequently** that users can interact with each job while it is running, creating **interactive computing**
 - Response time should be < 1 second
 - Each user has at least one program executing in memory (**process**)
- If several jobs ready to run at the same time (**CPU scheduling**)

Timer

- Timer to prevent infinite loop (or process hogging resources)
 - Timer is set to interrupt the computer after some time period
 - Keep a counter that is decremented by the physical clock
 - Operating system set the counter (privileged instruction)
 - When counter zero generate an interrupt
 - Set up before scheduling process to regain control or terminate program that exceeds allotted time

Dual-mode Operation

- **Dual-mode** operation allows OS to protect itself and other system components
 - **User mode** and **kernel mode** (supervisor mode, system mode, privileged mode)
e.g. 存取I/O 之類的行為, 由OS統一管理
- Mode bit provided by hardware
 - Provides ability to distinguish when system is running user code or kernel code.
- How do we guarantee that user does not explicitly set the mode bit to “kernel”?
 - System call changes mode to kernel, return from call resets it to user
- Some instructions designated as **privileged**, only executable in kernel mode
 - **The CPU is designed to cooperate with the OS**

Dual-mode Operation

The separation of User Mode and Kernel Mode is fundamental to modern operating system design for several critical reasons:

System Security

Isolation and Protection: Prevents malicious or buggy applications from directly manipulating hardware or corrupting the system, ensuring system stability. If applications could directly access hardware, a single program error could crash the entire system.

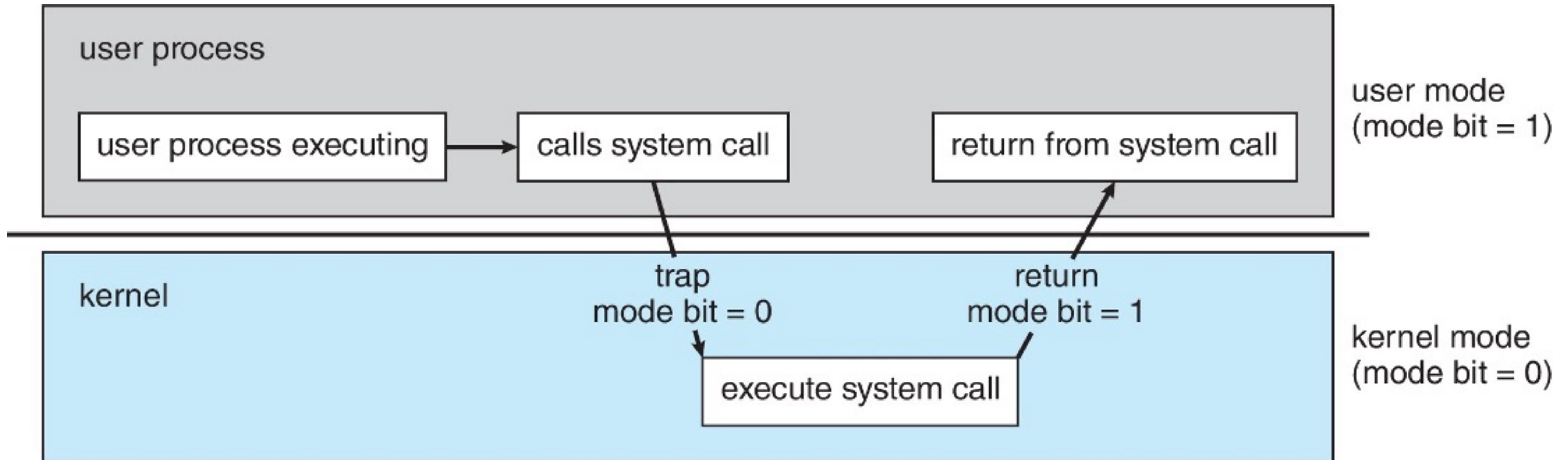
Access Control: Restricts applications to only use authorized resources, preventing unauthorized system access and maintaining system integrity.

Controlled System Access

Privileged Operations: Critical system operations like memory management, interrupt handling, and hardware control are restricted to kernel mode, ensuring these operations are performed safely and correctly.

System Call Interface: Provides a controlled mechanism for user programs to request system services, allowing the OS to validate requests and maintain system security.

Transition from User to Kernel Mode



Outline

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Operations
- Resource Management
- Virtualization

Resource Management

- Progress Management 会考
- Memory Management
- File-System Management
- Mass-Storage Management
- Cache Management
- I/O System Management

Process Management

- A process is a program in execution. It is a unit of work within the system. Program is a **passive entity**; **process** is an **active entity**.
Program是一个存在于Disk中的被动实体
- Process needs resources to accomplish its task
 - CPU, memory, I/O, files
- Process termination requires reclaim of any reusable resources
- Single-threaded process has one **program counter** specifying location of next instruction to execute
 - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically system has many processes, some user, some operating system running concurrently on one or more CPUs
 - Concurrency by multiplexing the CPUs among the processes / threads

Process Management

- The operating system is responsible for the following activities in connection with process management
 - Creating and deleting both user and system processes
 - Suspending and resuming processes
 - Providing mechanisms for process [synchronization](#)
 - Providing mechanisms for process communication
 - Providing mechanisms for [deadlock handling](#)

Memory Management

- To execute a program all (or part) of the instructions must be in memory
- All (or part) of the data that is needed by the program must be in memory
- Memory management determines what is in memory
- Memory management activities
 - Keeping track of which parts of memory are currently being used and by whom
 - Deciding which processes (or parts thereof) and data to move into and out of memory
 - Allocating and deallocating memory space as needed

File-system Management

- OS provides uniform, logical view of information storage
 - Abstracts physical properties to logical storage unit - file
- File-System management
 - Files usually organized into directories
 - Access control on most systems to determine who can access what
 - OS activities include
 - Creating and deleting files and directories
 - Manipulating files and directories
 - Mapping files onto secondary storage
 - Backing up files onto stable (non-volatile) storage media

Mass-Storage Management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- OS activities
 - Mounting and unmounting
 - Free-space management
 - Storage allocation
 - Disk scheduling
 - Reference: <https://www.geeksforgeeks.org/disk-scheduling-algorithms/>
 - Partitioning
 - Protection

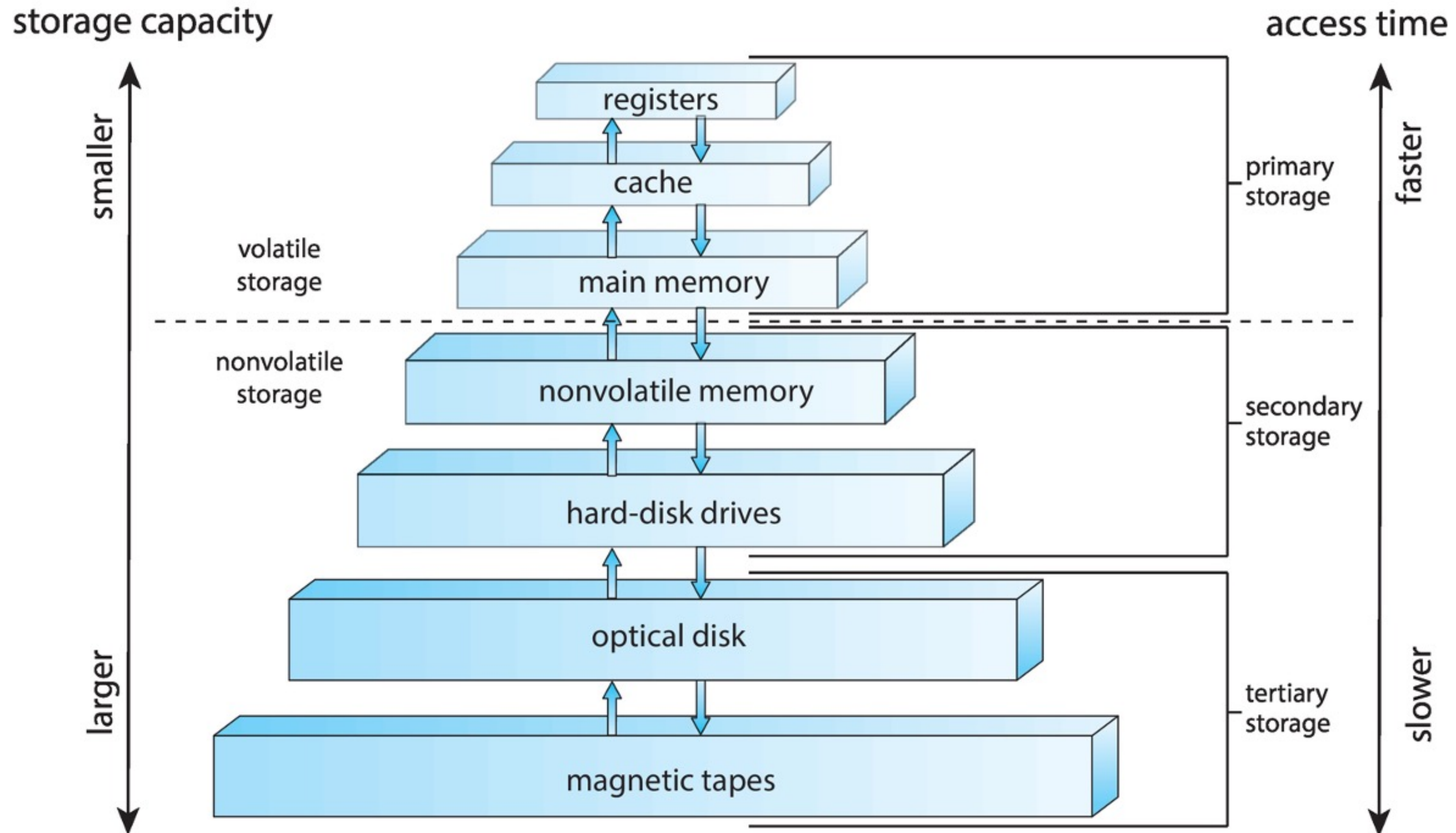
Caching

- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
 - If it is, information used directly from the cache (fast)
 - If not, data copied to cache and used there
- Cache smaller than storage being cached
 - Cache size and replacement policy

Characteristics of Various Types of Storage

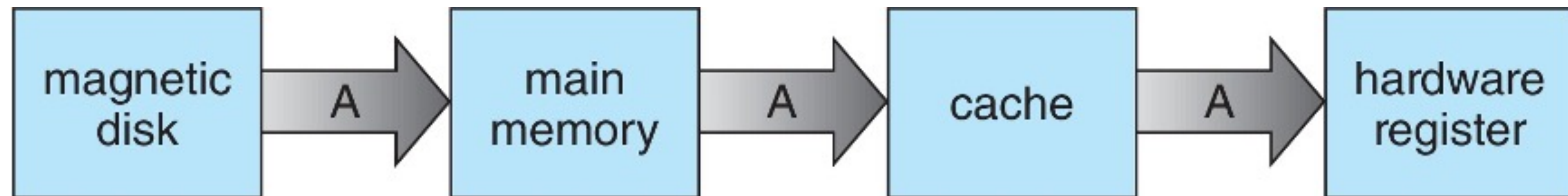
Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

Storage-device Hierarchy



Migration of Data 'A' from Disk to Register

- Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy
- Multiprocessor environment must provide **cache coherency** in hardware such that all CPUs have the most recent value in their cache



Cache Coherency

What is Cache Coherency?

Definition:

Cache coherency (or cache coherence) refers to a consistency mechanism in multiprocessor systems to ensure that **all caches reflect the most recent value of a shared data item**. It ensures that if multiple caches store copies of the same memory location, any updates to that location are properly reflected across all caches.

Why is Cache Coherency Needed?

In a multiprocessor system:

- Each processor/core often has its own private cache to reduce memory access latency.
- If two or more caches hold the same memory address, and one processor updates its value, other processors **might still read the stale (old) value** from their own caches unless coherency is enforced.

Outline

- What Operating Systems Do
- Computer-System Organization
- Computer-System Architecture
- Operating-System Operations
- Resource Management
- Virtualization

Virtualization

- Allows operating systems to run applications within other OSes
 - Vast and growing industry
- **Emulation** used when source CPU type different from target type (i.e. PowerPC to Intel x86) \ 模擬: 模擬一顆CPU的架構
 - Generally slowest method
 - When computer language not compiled to native code - Interpretation
- **Virtualization** - OS natively compiled for CPU, running guest OSes also natively compiled \ 虛擬化: 模擬一個OS環境
 - Consider VMware running WinXP guests, each running applications, all on native WinXP host OS
 - VMM (virtual machine manager) provides virtualization services

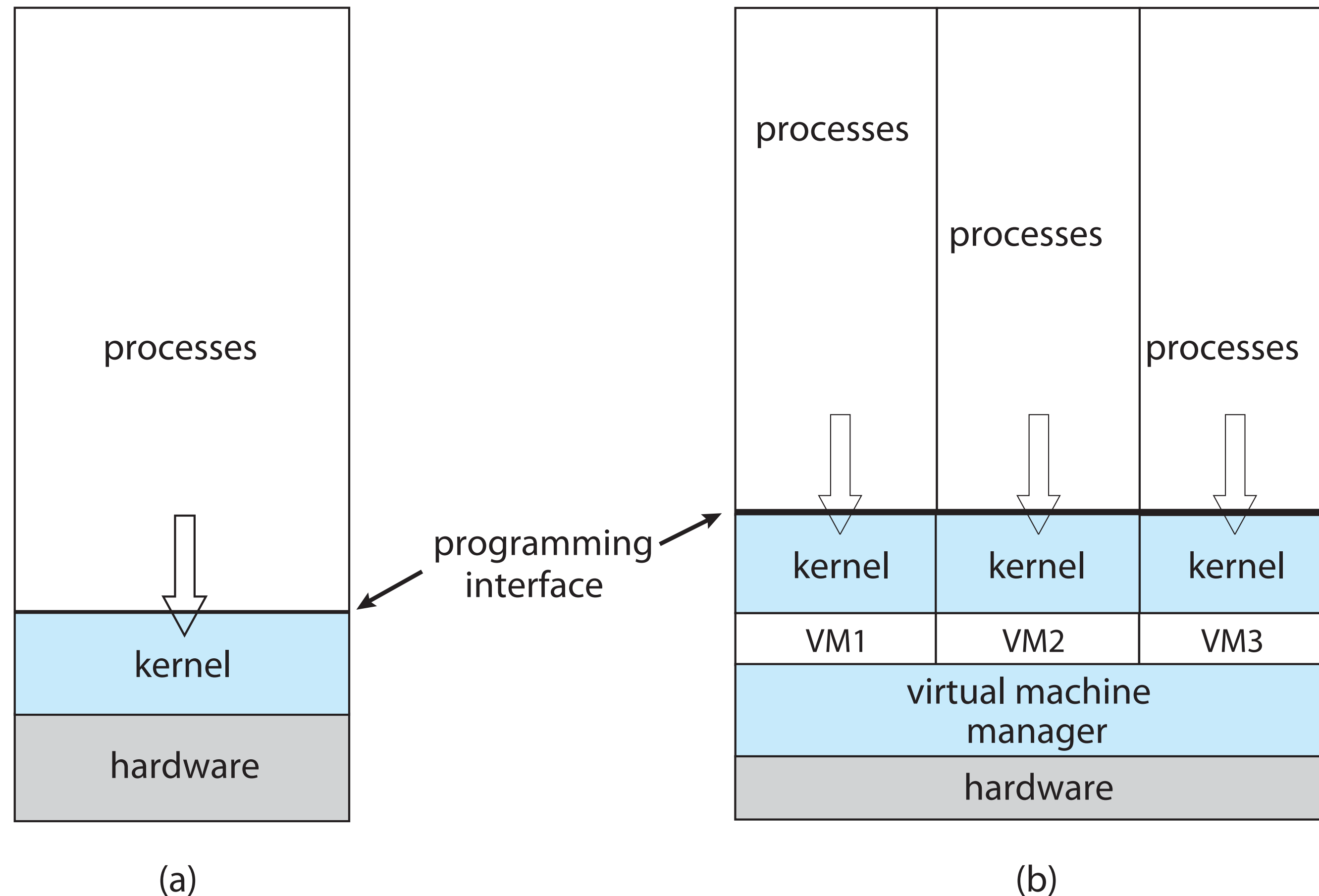
Virtualization

- Use cases involve laptops and desktops running multiple OSes for exploration or compatibility
 - Apple laptop running Mac OS X **host**, Windows as a **guest**
 - Reference: <https://www.arthurtoday.com/2009/12/host-guest.html>
 - Developing apps for multiple OSes without having multiple systems
 - Quality assurance testing applications without having multiple systems
 - Executing and managing compute environments within data centers

Virtualization

*傳統的虛擬化技術

- A computer running (a) a single operating system and (b) three virtual machines



OS-level Virtualization

*輕量、較現代的虛擬化技術

- OS-level virtualization is an operating system (OS) virtualization paradigm in which the kernel allows the existence of multiple isolated user space instances
- Docker is a set of platform as a service (PaaS) products that use OS-level virtualization to deliver software in packages called containers.
- Reference: <https://www.nakivo.com/blog/docker-vs-kubernetes/>

Practice

1. Download the Ubuntu ISO image from <https://www.ubuntu.com/>
2. Instructed the virtual machine software VirtualBox to use the ISO as the bootable medium and booted the virtual machine
3. Answered the installation questions and then installed and booted the operating system as a virtual machine

Download

- VirtualBox (Version 7.1.6)
 - <https://www.virtualbox.org/>
- Download Ubuntu (Version 24.04.2 LTS)
 - <https://ubuntu.com/download/desktop>
- Create a virtual machine and install Ubuntu
- **April, 2025 (Host OS: Windows)**

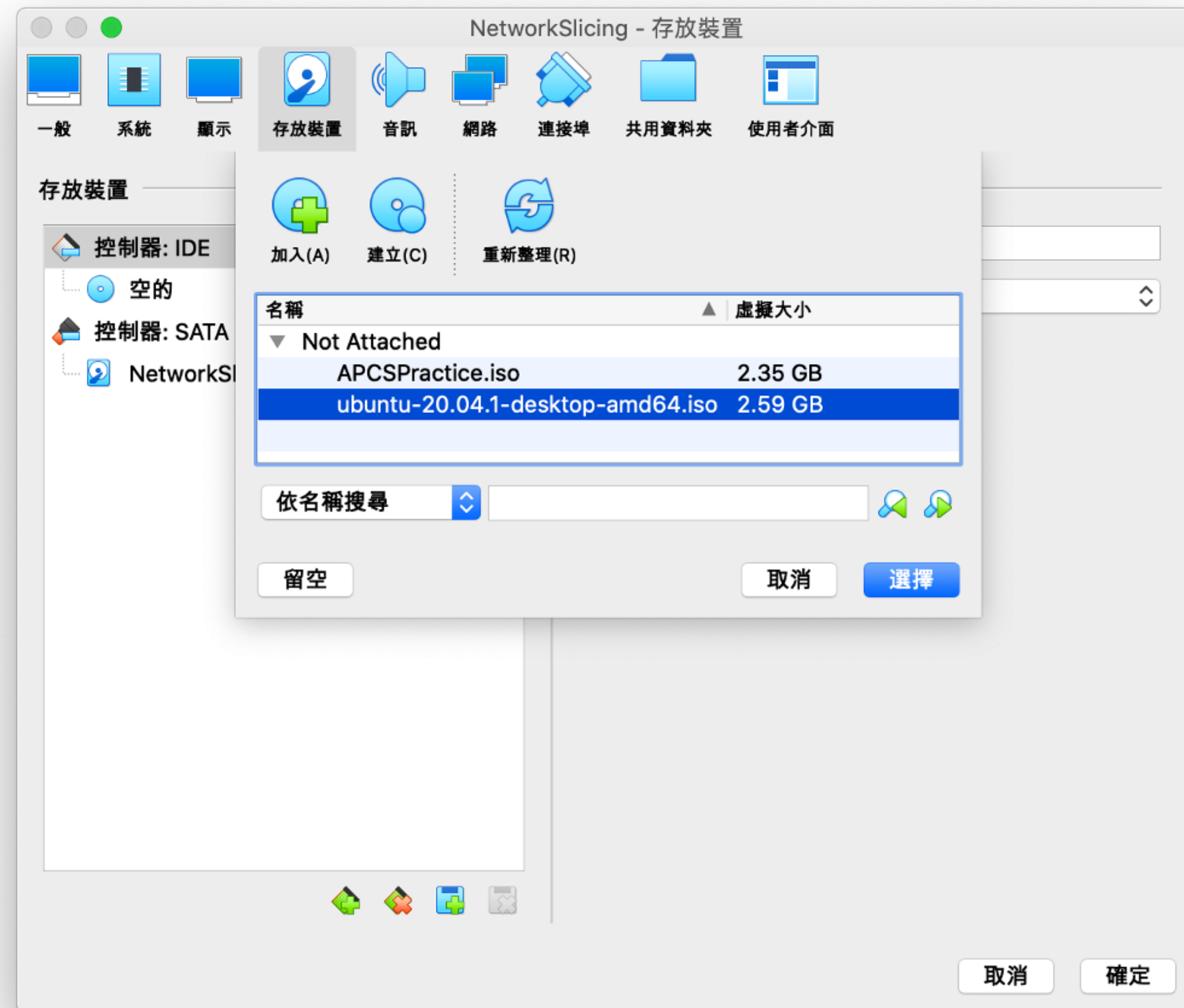
Download (2022)

- VirtualBox (Version 6.1.32)
 - <https://www.virtualbox.org/>
- Download Ubuntu (Version 20.04.4 LTS)
 - <https://ubuntu.com/download/desktop>
- Create a virtual machine and install Ubuntu

Create a virtual machine



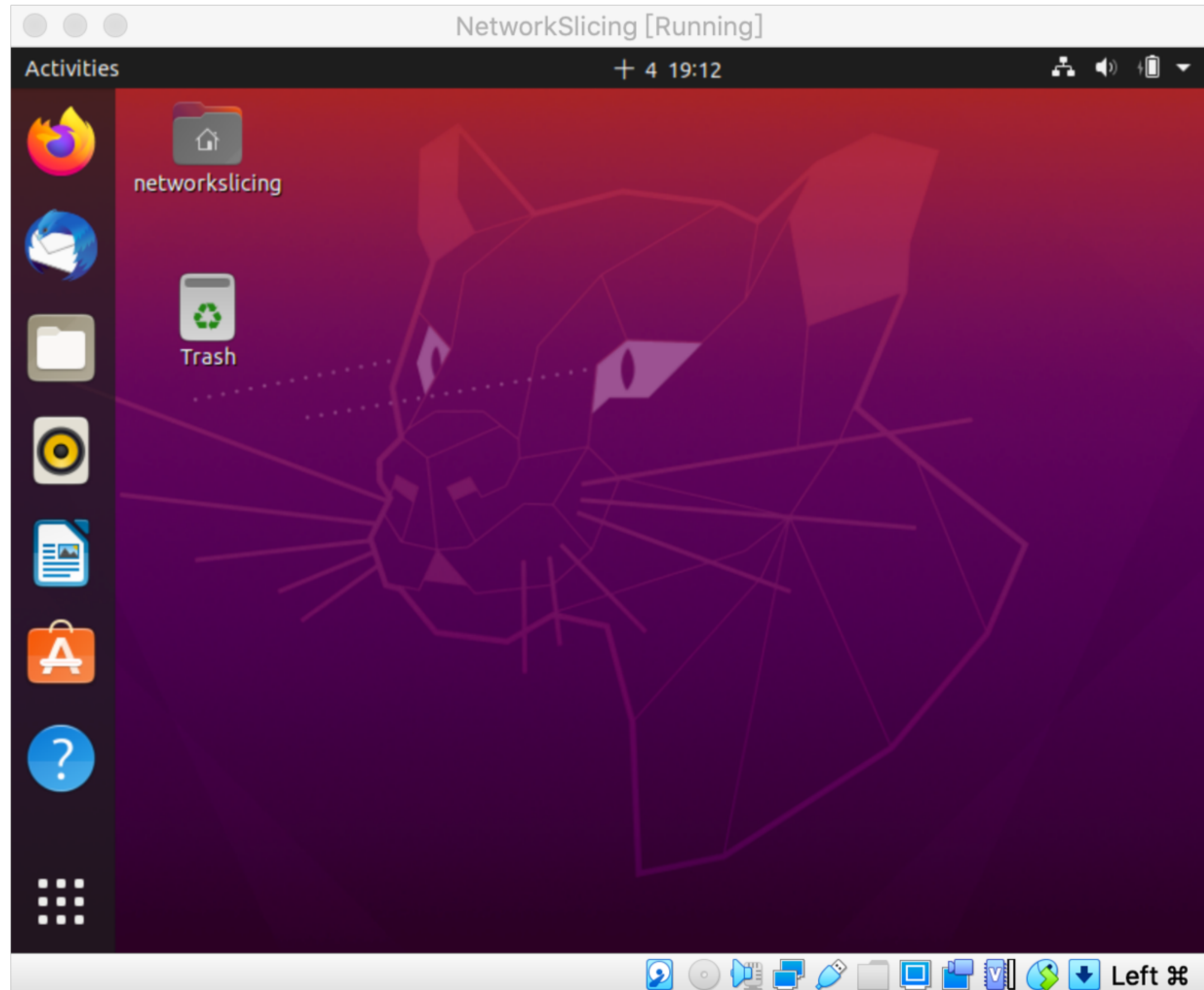
Bootable Medium



Boot the virtual machine



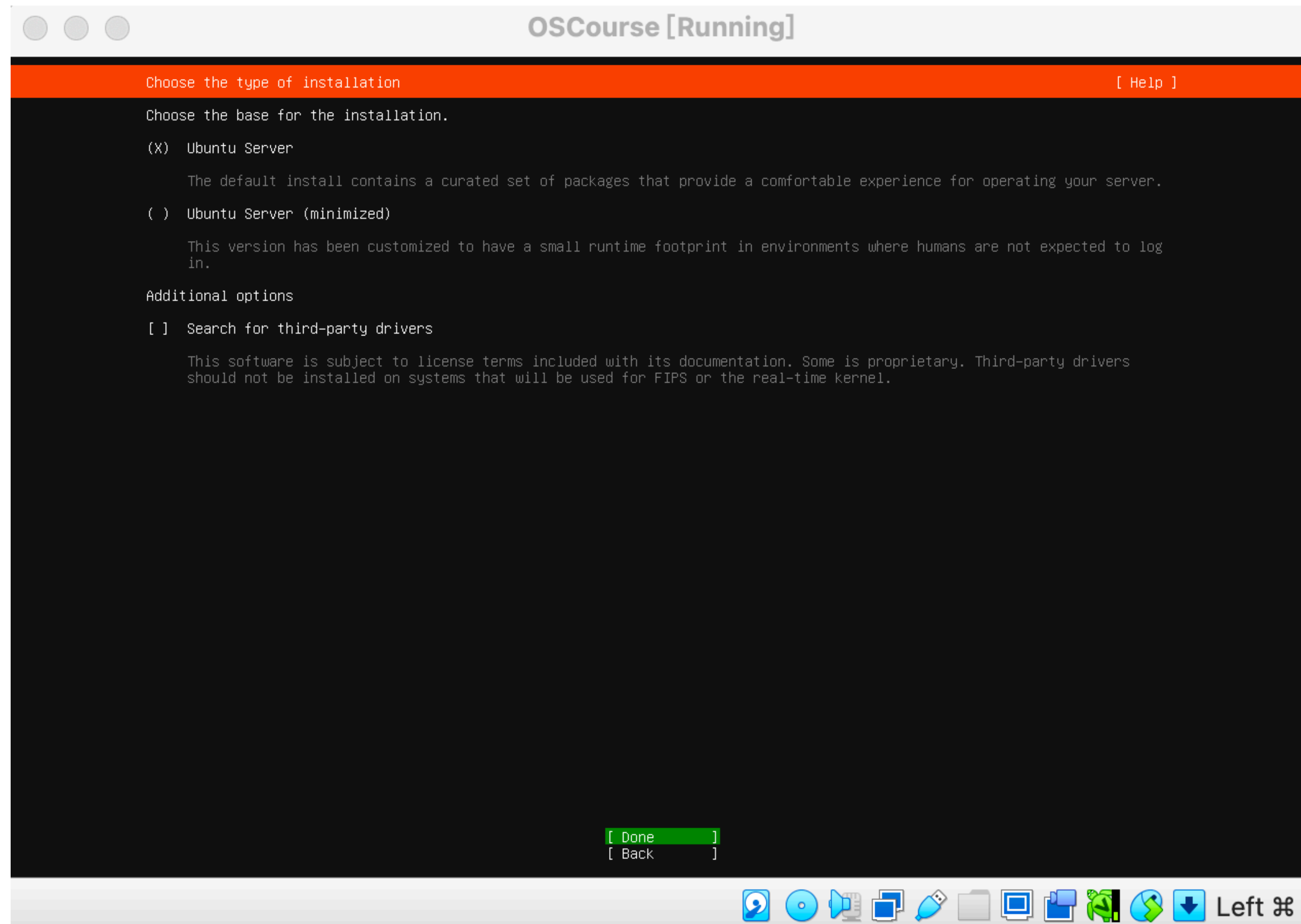
Complete



Download (2025)

- VirtualBox (Version 7.1.6)
 - <https://www.virtualbox.org/>
- Download Ubuntu for **ARM** (Version 24.04.2 LTS)
 - <https://ubuntu.com/download/server/arm>
- Create a virtual machine and install Ubuntu

Ask Questions



Complete

```
OSCourse [Running]

lincyu@lincyu:~$
lincyu@lincyu:~$ pwd
/home/lincyu
lincyu@lincyu:~$ ls
lincyu@lincyu:~$ ls -a
.  ..  .bash_logout  .bashrc  .cache  .profile
lincyu@lincyu:~$
Mounting snap-core22-1912.mount - Mount unit for core22, revision 1912...
[ OK ] Mounted snap-core22-1912.mount - Mount unit for core22, revision 1912.
Mounting snap-docker-3065.mount - Mount unit for docker, revision 3065...
[ OK ] Mounted snap-docker-3065.mount - Mount unit for docker, revision 3065.
[ OK ] Started snap.docker.hook.install-b140574d-e180-46bd-9e4f-0623a55d5472.scope.
Starting snap.docker.nvidia-container-toolkit.service - Service for snap application docker.nvidia-container-toolkit...
[ OK ] Finished snap.docker.nvidia-container-toolkit.service - Service for snap application docker.nvidia-container-toolkit.
[ OK ] Started snap.docker.dockerd.service - Service for snap application docker.dockerd.
[ OK ] Started snap.docker.hook.configure-2c01f56d-b6d6-4023-9e44-75921369874e.scope.
[ 60.581759] overlayfs: missing 'lowerdir'
Starting snap.docker.nvidia-container-toolkit.service - Service for snap application docker.nvidia-container-toolkit...
[ OK ] Finished snap.docker.nvidia-container-toolkit.service - Service for snap application docker.nvidia-container-toolkit.
Mounting snap-core24-741.mount - Mount unit for core24, revision 741...
[ OK ] Mounted snap-core24-741.mount - Mount unit for core24, revision 741.
Mounting snap-mosquitto-944.mount - Mount unit for mosquitto, revision 944...
[ OK ] Mounted snap-mosquitto-944.mount - Mount unit for mosquitto, revision 944.
[ OK ] Started snap.mosquitto.mosquitto.service - Service for snap application mosquitto.mosquitto.
[ 80.373223] cloud-init[1043]: Generating locales (this might take a while)...
[ 81.418318] cloud-init[1043]:   en_US.UTF-8... done
[ 81.422112] cloud-init[1043]: Generation complete.
[ OK ] Finished cloud-config.service - Cloud-init: Config Stage.
Starting cloud-final.service - Cloud-init: Final Stage...
[ 81.635031] cloud-init[3143]: Cloud-init v. 24.4-0ubuntu1~24.04.2 running 'modules:final' at Thu, 17 Apr 2025 01:03:08 +0000. Up 81.61 seconds.
ci-info: no authorized SSH keys fingerprints found for user lincyu.
<14>Apr 17 01:03:08 cloud-init: #####
<14>Apr 17 01:03:08 cloud-init: -----BEGIN SSH HOST KEY FINGERPRINTS-----
<14>Apr 17 01:03:08 cloud-init: 256 SHA256:vKPGvYbAAdtyWdDorUHxZY3pJ0DXLmJ07Ab00V8WBVrA root@lincyu (ECDSA)
<14>Apr 17 01:03:08 cloud-init: 256 SHA256:I42kvYD2a0dsD5R2C5MfQFVyyus9RYOKIniUPCeCNx4 root@lincyu (ED25519)
<14>Apr 17 01:03:08 cloud-init: 3072 SHA256:tIYoeijB10NH25eorEBkVI/W0K8KJmRLycF3Jx/M4RI root@lincyu (RSA)
<14>Apr 17 01:03:08 cloud-init: -----END SSH HOST KEY FINGERPRINTS-----
<14>Apr 17 01:03:08 cloud-init: #####
-----BEGIN SSH HOST KEY KEYS-----
ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBBHB2B2t3LQ/jaKpKj7IaA1rBhdp2lAbyXWFEA+YpBqXKLJ3NU0ux0FvH9zY28ztgEX83B9nfhP4xK2jDq+9DzznM=
root@lincyu
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBtQk9jccmf2hkI+l44QF+Ay92hYoL9ksDciEEwKgJJq root@lincyu
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQCJsvG6+jP+tFca65Q5J6W8oSCk4VjK2v54/mhgqKUJ/g6qRHJGfXrcKLsqLvpvXhPaG7/P2GMbB2kaSzmKxd5dbW2g8NbDySPJBr+tpYNBDEJDTs3/q+v7tTJa
PTuRW3VIarY5Hz/ntvLVdty7V0VpqbLg7aXixDHdwbLtlba2PuPUCWHzxSeb5yduLtbKdEEJU4PHDe5xpkWAEAy69pN1yMR61umeS098FOnULjNn1lg1BNpHnf+BRSarCw2pS1hNORfM5vd+acnY20H7Jr5Hg/
EyBIQ1p+D3wnQvTLJ2D3a0nh6A6nKT741NKkbJUCGnp6v3aMb7KEoP0peG2M5oHiHkxLTldVc9idrY14T4596PD59iKztRWFUc34kWNaln1JsJkFVUxuxECTtJDFGI/+If1U1D/ItAF7Q1oA2jRWLVYqun5G4Wmb
j0akOSXI+IoMae4qH1KXIiXU2EYTpIXTeBCrr6Ws0eN1+jX2IhHaafVqK62IPE2ET6TGQ5k= root@lincyu
-----END SSH HOST KEY KEYS-----
[ 81.677577] cloud-init[3143]: Cloud-init v. 24.4-0ubuntu1~24.04.2 finished at Thu, 17 Apr 2025 01:03:08 +0000. Datasource DataSourceNone. Up 81.67 seconds
[ OK ] Finished cloud-final.service - Cloud-init: Final Stage.

lincyu@lincyu:~$
```

Other Solutions

- Windows Subsystem for Linux (WSL)
 - Reference: <https://bayareanotes.com/wsl-installation/>
- Cygwin
 - <https://www.cygwin.com/>

Summary

会考

- What Operating Systems Do
 - Easy to use
 - Resource Allocation
- Computer-System Organization
 - Interrupt / Context Switch
- Computer-System Architecture
 - Symmetric MultiProcessing (SMP) / NUMA (Non-Uniform Memory Access)
- Operating-System Operations
 - Timesharing
 - Process

Summary

- Resource Management
 - Process Management
 - Memory Management
- Virtualization
 - Virtual Machine
 - OS-level Virtualization